my-agents-cli

An enterprise-ready ReAct agent project scaffolded with `google-agents-cli` version `1.13.0`.
It features a FastAPI-based backend, A2A (Agent-to-Agent) communication, Vertex AI Reasoning Engine route adapter, automated test suites (unit, integration, evaluation), and Terraform infrastructure as code for GCP deployment.

---

1. Project Detailed Directory Structure and Component Analysis

```
my-agents-cli/
├── 📁 app/                                # [Initial] Core agent application code
│   ├── 📄 agent.py                        # 🟢 [Initial] Main agent logic and Mock tools (weather,
current time)
│   ├── 📄 fast_api_app.py                 # 🟢 [Initial] FastAPI backend server supporting REST / SSE /
A2A
│   ├── 📄 __init__.py                     # 🟢 [Initial] Python package initialization file
│   ├── 📁 app_utils/                      # 🟢 [Initial] Agent auxiliary utility modules
│   │   ├── 📄 a2a.py                      # 🟢 [Initial] A2A protocol communication & Agent Card
handling
│   │   ├── 📄 reasoning_engine_adapter.py # 🟢 [Initial] Vertex AI Reasoning Engine route adapter
│   │   └── 📄 services.py                 # 🟢 [Initial] ADK session and artifact service factory
│   └── 📁 .adk/                           # ⚡ [Runtime Generated] Internal local cache folder
│       └── 📄 session.db                  # ⚡ [Runtime Generated] Local SQLite DB storing session and
conversation state
│
├── 📁 deployment/                         # ⚡ [scaffold enhance] Infrastructure and deployment code
directory
│   └── 📁 terraform/                      # ⚡ [Dynamically Generated] Terraform HCL code for GCP
resource deployment
│       ├── 📁 shared/                     # ⚡ [Dynamically Generated] Shared infrastructure modules
(BigQuery log schema, etc.)
│       └── 📁 single-project/             # ⚡ [Dynamically Generated] Single-project deployment scripts
(service.tf, iam.tf, etc.)
│
├── 📁 tests/                              # 🟢 [Initial] Testing and evaluation pipeline
│   ├── 📁 unit/                           # 🟢 [Initial] Unit test directory
│   │   └── 📄 test_dummy.py               # 🟢 [Initial] Basic dummy unit test
│   ├── 📁 integration/                    # 🟢 [Initial] Server integration test directory
│   │   ├── 📄 test_agent.py               # 🟢 [Initial] Agent logic integration test
│   │   └── 📄 test_server_e2e.py          # 🟢 [Initial] FastAPI and SSE endpoint E2E test
│   └── 📁 eval/                           # 🟢 [Initial] Agent evaluation pipeline
│       ├── 📄 eval_config.yaml            # 🟢 [Initial] Evaluation configuration and metrics
specification
│       ├── 📄 response_quality.py         # 🟢 [Initial] LLM response quality automated scorer
│       └── 📁 datasets/                   # 🟢 [Initial] Evaluation datasets folder
│           ├── 📄 basic-dataset.json      # 🟢 [Initial] Evaluation query and ground-truth answer data
│           └── 📄 README.md               # 🟢 [Initial] Dataset guidelines
│
├── 📁 .venv/                              # ⚡ [uv sync / install] uv-based Python virtual environment
├── 📁 .google-agents-cli/                 # ⚡ [Runtime Generated] Local server execution logs directory
│   ├── 📄 run_server.log                  # ⚡ [Dynamically Generated] Server standard output log
│   └── 📄 run_server.stderr.log           # ⚡ [Dynamically Generated] Server error log
│
├── 📄 agents-cli-manifest.yaml            # 🟢 [Initial] agents-cli project metadata specification
├── 📄 pyproject.toml                      # 🟢 [Initial] uv-based project dependency and package
settings
├── 📄 uv.lock                             # ⚡ [Package Install] Exact package version lockfile
├── 📄 deployment_metadata.json            # ⚡ [Deployment] Remote GCP engine ID and deployment history
├── 📄 Dockerfile                          # 🟢 [Initial] Dockerfile for containerized build and
deployment
├── 📄 .env                                # 🟢 [Initial / Config] Local environment variable file (API
keys, etc.)
├── 📄 .env.example                        # 🟢 [Initial] Environment settings template
├── 📄 .gitignore                          # 🟢 [Initial] Git exclusion rules
├── 📄 .gcloudignore                       # 🟢 [Initial] Google Cloud build exclusion rules
├── 📄 GEMINI.md                           # 🟢 [Initial] Project context guide for AI assistants
├── 📄 README.md                           # 📄 [English Guide] Standard project guide
└── 📄 README-KR.md                        # 📄 [Korean Guide] Detailed Korean guide
```

2. Requirements

Before starting, make sure you have:

- **uv**: Fast Python package manager - Installation Guide (add packages with `uv add <package>`)
- **agents-cli**: Google Agents CLI - Install via `uv tool install google-agents-cli`
- **Google Cloud SDK**: For Google Cloud services - Installation Guide

3. How to Run

Execute the following commands from within the `my-agents-cli` directory:

```
# Navigate to directory
cd my-agents-cli

# 1. Install dependency packages
agents-cli install

# 2. Launch local interactive playground web UI (default port: 8000)
agents-cli playground

# 3. Directly launch FastAPI server with Uvicorn
uv run uvicorn app.fast_api_app:app --host 0.0.0.0 --port 8000 --reload

# 4. Run code quality checks (Linting)
agents-cli lint

# 5. Run unit and integration tests
uv run pytest tests/unit tests/integration

# 6. Run agent evaluation (Eval) pipeline
agents-cli eval

# 7. Deploy to Google Cloud Agent Runtime
gcloud config set project <your-project-id>
agents-cli deploy

# 8. Register deployed agent with Gemini
agents-cli publish gemini
```

4. 🛠️ Project Management Commands

Command	Action
`agents-cli scaffold enhance`	Adds CI/CD pipelines and Terraform infrastructure code
`agents-cli infra cicd`	One-command setup for entire CI/CD pipeline and infrastructure
`agents-cli scaffold upgrade`	Upgrades template to the latest version while preserving user customizations

5. Request Processing Flow: Uvicorn to Agent

Based on `app/fast_api_app.py`, this section explains how an HTTP request sent by a client reaches `root_agent`.

1) Server Boot

```
uvicorn app.fast_api_app:app --host 0.0.0.0 --port 8000
```

- Reads `.env` using `load_dotenv()` and checks `ALLOW_ORIGINS` and `GOOGLE_CLOUD_AGENT_ENGINE_ENABLE_TELEMETRY`.
- Creates the FastAPI app using `get_fast_api_app(...)` (provided by ADK). Since `web=True`, both the ADK Web UI and API routes are registered.
- Shared service instances are configured via `shared://session` and `shared://artifact` URIs in `app/app_utils/services.py`.
- Registers `/api/reasoning_engine` routes via `attach_reasoning_engine_routes(app)`.

2) Lifespan: Server Startup Event

The `lifespan` function executes once when the server starts up:

- Imports `adk_app` and `root_agent` from `app.agent`.
- `Runner(app=adk_app, session_service=..., artifact_service=..., auto_create_session=True)` and stores it in `app.state.runner`.
- Attaches A2A routes at `/a2a/app` via `attach_a2a_routes(...)`.

3) Route Dispatch

Incoming Request	Origin	Handler / Responsibility
`POST /run_sse`	ADK (`api_server.py`)	Primary streaming SSE endpoint used by ADK Web UI and `agents-cli playground`
`POST /run`	ADK	Executes agent and returns all accumulated events as a single JSON array
`POST /a2a/app` (JSON-RPC)	`app_utils/a2a.py`	`A2aAgentExecutor(runner=...)` receives requests and executes Runner
`POST /api/reasoning_engine`	`app_utils/reasoning_engine_adapter.py`	Invokes `adk_app` methods for Google Cloud Console playground

4) `/run_sse` Internal Flow

```
HTTP POST /run_sse (JSON payload)
  |
  ▼
api_server.run_agent_sse()
  |  get_runner_async("app")
  |    └─ Retrieves cached Runner instance
  ▼
Runner.run_async(user_id, session_id, new_message)
  |  1) Query session from session_service (auto-create if missing)
  |  2) Append new_message as a session event
  |  3) Run plugin hooks and execute root_agent
  ▼
root_agent (LlmAgent "my_agents_cli")
  |  Calls Gemini combining instructions and registered tools (get_weather, get_current_time)
  |  ├─ When tool call requested → executes tool locally → feeds result back to Gemini (loop)
  |  └─ When final text generated → finishes response
  ▼
Stream each Event to client as an SSE ("data: {...}\n\n") chunk
```

Key points:

- **Routes are thin**: The FastAPI route merely validates the request and hands it to the Runner. The actual reasoning loop runs inside ADK's `Runner` and `LlmAgent`.
- **Runner is reused per application**: `/run_sse` uses the Runner cached internally by ADK, while `app.state.runner` created in `lifespan` is stored in application state.

---

6. Observability and A2A Support

- **Observability**: Built-in telemetry exports trace, metrics, and logs to Cloud Trace, BigQuery, and Cloud Logging.
- **A2A Inspector**: This agent fully supports the A2A Protocol. Use A2A Inspector to test interoperability.